

Set up your JavaScript dev environment

From a blank Windows machine to writing and running your first JavaScript program — no prior experience needed.

01 • Node.js

02 • VS Code

03 • Terminal

04 • First program

05 • npm basics

06 • Git

07 • Next steps

01 Install Node.js

JavaScript originally ran only inside browsers. **Node.js** lets it run directly on your computer — which means you can write programs, build servers, and use thousands of tools that the JS ecosystem provides.

- 1 **Go to nodejs.org** and click the button labelled **LTS** (Long-Term Support). This is the stable version — use it unless a tutorial tells you otherwise.
- 2 **Run the installer** you downloaded. Keep all defaults and click *Next* through every screen. Make sure *"Add to PATH"* is ticked.
- 3 **Restart your computer** after the installer finishes.
- 4 **Verify it worked.** Open **Command Prompt** (`Win + R` , type `cmd` , press Enter) and run:

```
node --version
npm --version
```

You should see version numbers like `v20.x.x` and `10.x.x` . If you do, Node.js is ready.

 **What is npm?** It stands for *Node Package Manager*. It installs automatically with Node and lets you add ready-made code libraries to your projects. You'll use it in Step 5.

02 Install VS Code

A code editor is where you write your programs. **Visual Studio Code** (VS Code) is free, beginner-friendly, and the most popular choice for JavaScript development.

- 1 Download VS Code from code.visualstudio.com. Choose the **Windows** installer.
- 2 Run the installer. On the *"Select Additional Tasks"* screen, tick **"Add to PATH"** and **"Open with Code"** for both file and folder options.
- 3 Launch VS Code after installation.

Recommended extensions — install these by clicking the puzzle-piece icon in the left sidebar:

Prettier

Automatically formats your code to look clean every time you save.

ESLint

Underlines mistakes in your code before you even run it.

GitLens

Shows who changed what in your files when you start using Git.

Error Lens

Shows error messages right on the line they occur — no hunting around.

03 Get comfortable with the terminal

The **terminal** (also called the command line) is a text-based way to control your computer. As a developer, you'll use it to run your programs, install packages, and manage your projects.

VS Code has a built-in terminal. Open it with `Ctrl + `` (the backtick key, below Escape). It opens at the bottom of the window.

The five commands you'll use most often:

```
# Show what folder you're currently in
pwd
```

```
# List files in the current folder
ls

# Move into a folder called "my-project"
cd my-project

# Move up one folder level
cd ..

# Create a new folder called "my-project"
mkdir my-project
```

⚠️ VS Code's default terminal on Windows is **PowerShell**. It works fine. If a tutorial shows slightly different commands, they may be writing for Mac/Linux (bash). The commands above work in both.

04 Write and run your first program

Let's make sure everything works end-to-end.

1 Create a project folder. In your terminal, run:

```
mkdir my-first-app
cd my-first-app
```

2 Open it in VS Code:

```
code .
```

The `.` means "open the current folder". VS Code will open with your new folder loaded.

3 Create a new file. In VS Code, press `Ctrl + N`, then `Ctrl + S` and save it as `index.js`.

4 Type this code into the file:

```
const name = "World";
console.log(`Hello, ${name}!`);

for (let i = 1; i <= 5; i++) {
```

```
console.log( `Number: ${i}` );  
}
```

5 Run it in the VS Code terminal:

```
node index.js
```

You should see `Hello, World!` followed by numbers 1 through 5 printed in the terminal. 🎉



`console.log()` is your best friend when learning. It prints any value to the terminal so you can see what your program is doing at any point.

05 npm basics – using packages

npm gives you access to over a million packages — reusable chunks of code that other developers have shared. Instead of writing everything yourself, you install a package and use what it provides.

Start every project by initialising it:

```
# Run this once inside your project folder  
npm init -y
```

This creates a `package.json` file — a record of your project's name, version, and every package it depends on.

Installing a package — for example, `chalk`, which adds colour to terminal output:

```
npm install chalk
```

This downloads the package into a `node_modules` folder and records it in `package.json`.

The `node_modules` folder

This folder can get large and is automatically regenerated — never manually edit it, and never commit it to Git. When you share your project, others run `npm install` to recreate it from your `package.json`.

`package-lock.json`

Created automatically alongside `package.json`. It locks the exact versions of every package. You should commit this file to Git so your project builds identically for everyone.

06 Install Git – save your work history

Git is version control: it tracks every change you make to your code, lets you undo mistakes, and makes it possible to collaborate with other developers.

1 Download Git from git-scm.com and run the installer. Accept all defaults.

2 Tell Git your name and email (used to label your changes):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

3 Initialise a Git repository inside your project:

```
git init
```

4 Create a `.gitignore` file in your project root and add this line to it:

```
node_modules/
```

This tells Git to ignore the `node_modules` folder.

5 Make your first commit:

```
# Stage all files
git add .

# Save a snapshot with a message
git commit -m "Initial commit"
```



Think of a **commit** as a save point in a game. You can always travel back to any previous commit. Make commits often, whenever you finish a small piece of working code.

07

What to learn next

Your environment is ready. Here's a clear path forward:

JavaScript fundamentals

Variables, functions, arrays, objects, loops. Start with javascript.info — it's free and thorough.

Browser DevTools

Press F12 in Chrome or Edge. The Console tab lets you run JS live in the browser.

GitHub

A home for your Git repositories online. Create a free account and push your projects there.

A first project

Build a simple to-do list app or a number guessing game. Struggle through it — that's where learning happens.

// you're set up

Node.js · VS Code · npm · Git — that's the complete foundation. Everything else you'll learn builds on top of this stack.